5220411079 - M Zafid Affan

Laporan Proyek REST API Firebase dengan Node.js dan Express

A. Pendahuluan

Proyek ini bertujuan untuk membangun REST API sederhana menggunakan Node.js dan Express, yang dapat menangani data laporan dari pengguna dan menyimpannya ke Firebase Realtime Database. Firebase Realtime Database dipilih karena kemampuannya untuk menyimpan dan menyinkronkan data antar pengguna secara real-time. Dengan backend ini, diharapkan data laporan dapat dikelola dengan cepat, efisien, dan aman.

API ini dirancang untuk digunakan dalam aplikasi pelaporan sederhana, seperti laporan kejadian, pelaporan lingkungan, atau laporan pengguna, di mana setiap entri memiliki identitas unik dan informasi dasar seperti nama dan email. Backend ini juga dapat diperluas lebih lanjut untuk mendukung operasi CRUD lainnya, serta diintegrasikan dengan aplikasi frontend berbasis web atau mobile. Proyek ini juga menjadi studi kasus yang bagus untuk memahami integrasi antara backend Express dan layanan cloud Firebase, yang merupakan keterampilan penting dalam pengembangan perangkat lunak modern.

B. Tujuan

Bagian ini menjelaskan tujuan utama dari pengembangan proyek REST API menggunakan Node.js dan Firebase Realtime Database. Tujuan-tujuan ini dirancang agar pengembang memahami prinsip dasar pembuatan backend sederhana yang dapat menangani permintaan HTTP, serta bagaimana data dapat disimpan dan dikelola secara real-time menggunakan layanan cloud. Selain itu, proyek ini juga bertujuan untuk memperkenalkan konsep middleware dalam Express, yang berperan penting dalam pengolahan data dan pengaturan akses dari klien.

- Membuat backend REST API sederhana untuk CRUD data laporan.
- Mengintegrasikan Firebase Realtime Database sebagai media penyimpanan data.
- Memahami penggunaan middleware Express untuk parsing JSON dan CORS.

C. Tools & Dependensi

Untuk membangun dan menjalankan proyek REST API ini, digunakan beberapa tools dan dependensi penting yang mendukung pengembangan backend modern. Pemilihan tools tersebut bertujuan untuk mempermudah proses pembuatan server, pengelolaan routing, serta koneksi ke layanan Firebase Realtime Database. Selain itu, dependensi seperti

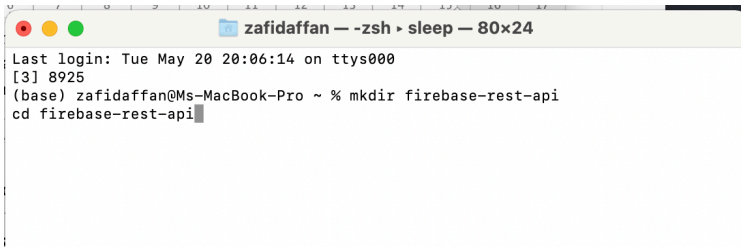
middleware CORS dan aplikasi Postman juga digunakan untuk memastikan API dapat diuji dan diakses dengan lancar dari berbagai platform.

- Node.js (v14 atau lebih baru)
- Express.js (Web framework)
- Firebase Admin SDK (Untuk akses Firebase Realtime Database)
- CORS Middleware (Untuk mengatur akses lintas domain)
- Postman (Untuk testing API)

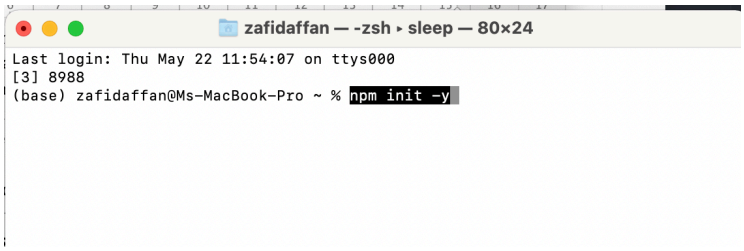
D. Langkah langkah

1. Install Node.js

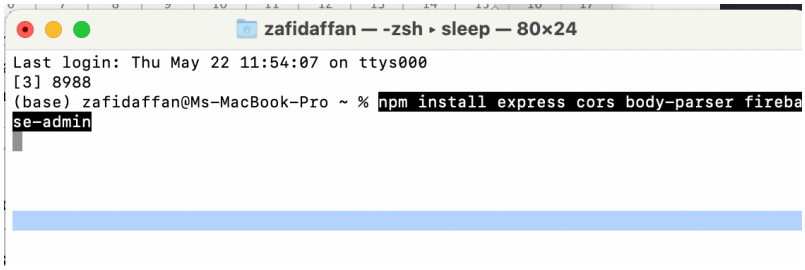
2. Buat folder proyek



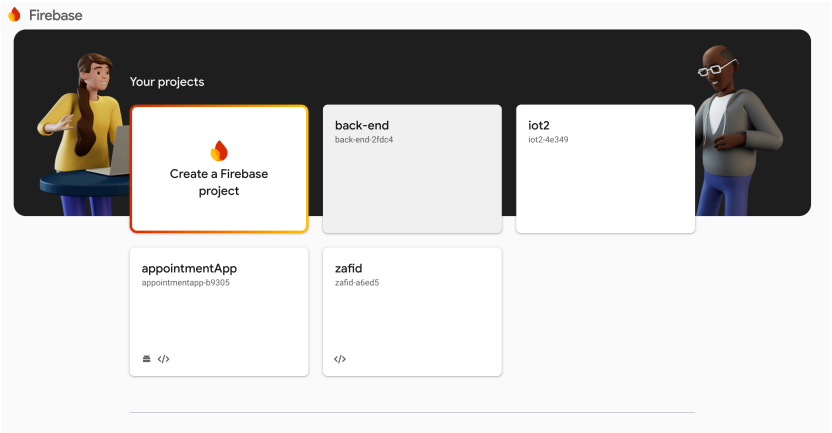
3. Inisialisasi Proyek Node.js



4. Install Paket yang Dibutuhkan

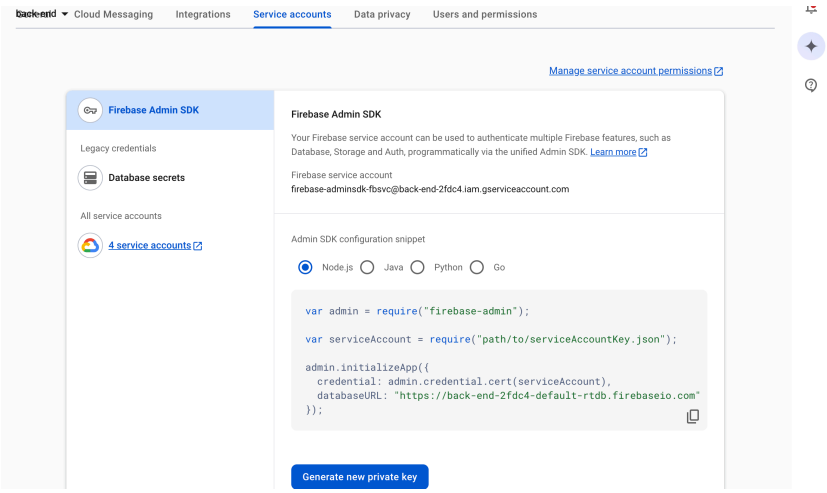


5. Buat Proyek di Firebase Console



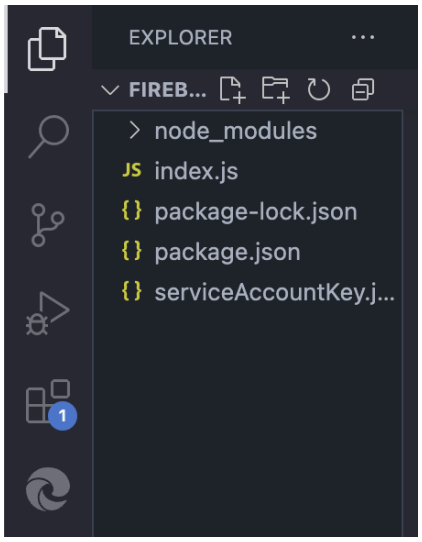
- Kunjungi https://console.firebase.google.com/
- Buat proyek baru (contoh: API Laporan Sampah)
- Aktifkan Cloud Firestore

6. Buat dan Unduh File Service Account



- Masuk ke Project Settings > Service accounts > klik Generate new private key
- Simpan file .json dan letakkan di folder proyek (misal: serviceAccountKey.json)

7. Struktur Project



Keterngan:

```
firebase-rest-api/
│
├── index.js            # File utama server Express
├── package.json        # File konfigurasi npm
├── serviceAccountKey.json  # File kredensial Firebase Admin SDK
└── node_modules/       # Folder dependensi npm
```

8. Isi file index.js

```
const express = require('express');
const cors = require('cors');
const admin = require('firebase-admin');
const app = express();
const PORT = 3000;

// Inisialisasi Firebase Admin
const serviceAccount = require('./serviceAccountKey.json');

admin.initializeApp({
  credential: admin.credential.cert(serviceAccount),
  databaseURL: 'https://back-end-2fdc4-default-rtdb.firebaseio.com/' // ganti
sesuai database kamu
});

const db = admin.database();

// Middleware
```

```
app.use(cors());
app.use(express.json());

// Test route
app.get('/', (req, res) => {
  res.send('API dengan Realtime Database berjalan!');
});

// Simpan laporan ke Realtime Database
app.post('/laporan', async (req, res) => {
  const { id, nama, email } = req.body;

  if (!id || !nama || !email) {
    return res.status(400).json({ message: 'Semua field (id, nama, email) wajib
diisi' });
  }

  try {
    const ref = db.ref('laporan/' + id); // Simpan berdasarkan ID
    await ref.set({ id, nama, email });

    res.status(200).json({ message: 'Laporan berhasil disimpan' });
  } catch (error) {
    console.error('Gagal menyimpan data:', error);
    res.status(500).json({ message: 'Terjadi kesalahan saat menyimpan laporan',
error: error.message });
  }
});

// Jalankan server
app.listen(PORT, () => {
  console.log(`Server berjalan di http://localhost:${PORT}`);
});
```

9. Jalankan server

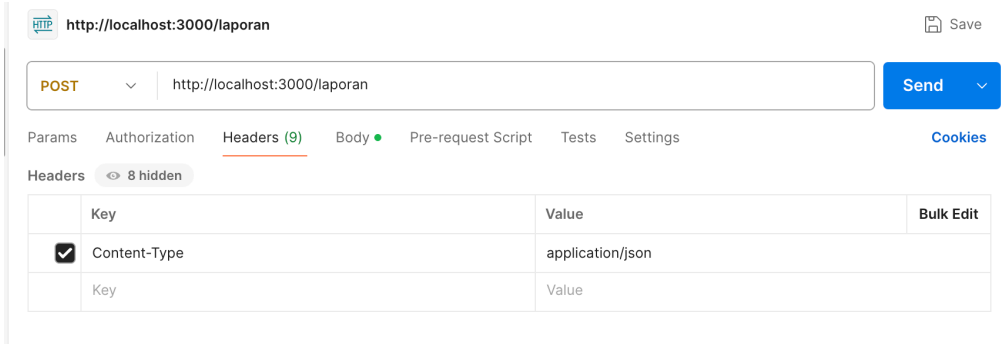
```
PROBLEMS    OUTPUT    DEBUG CONSOLE    PORTS    TERMINAL

[3] 9691
(base) zafidaffan@Ms-MacBook-Pro firebase-rest-api % node index.js
Server berjalan di http://localhost:3000
```

E. Uji API

1. Menggunakan Postman dengan methode Post dengan URL http://localhost:3000/laporan

2. Set Header Content-Type

Di tab Headers tambahkan:

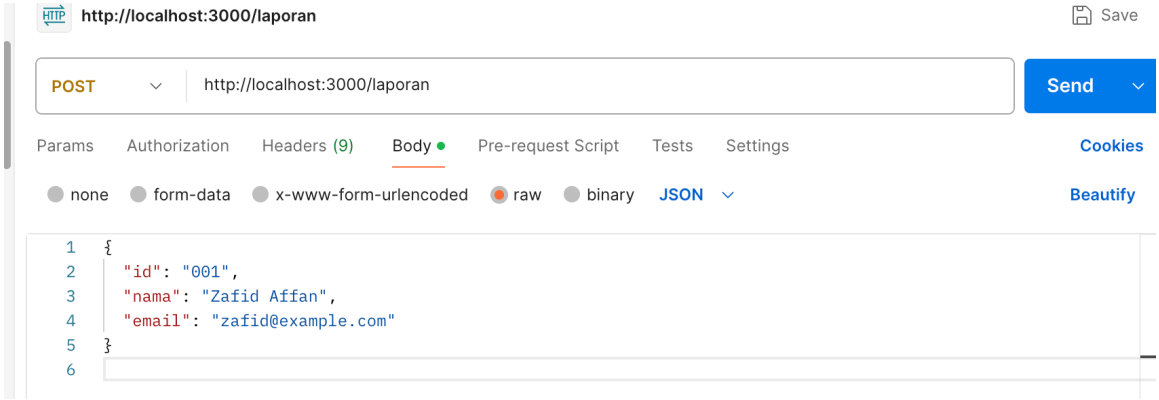


3. Isi Body dengan format JSON

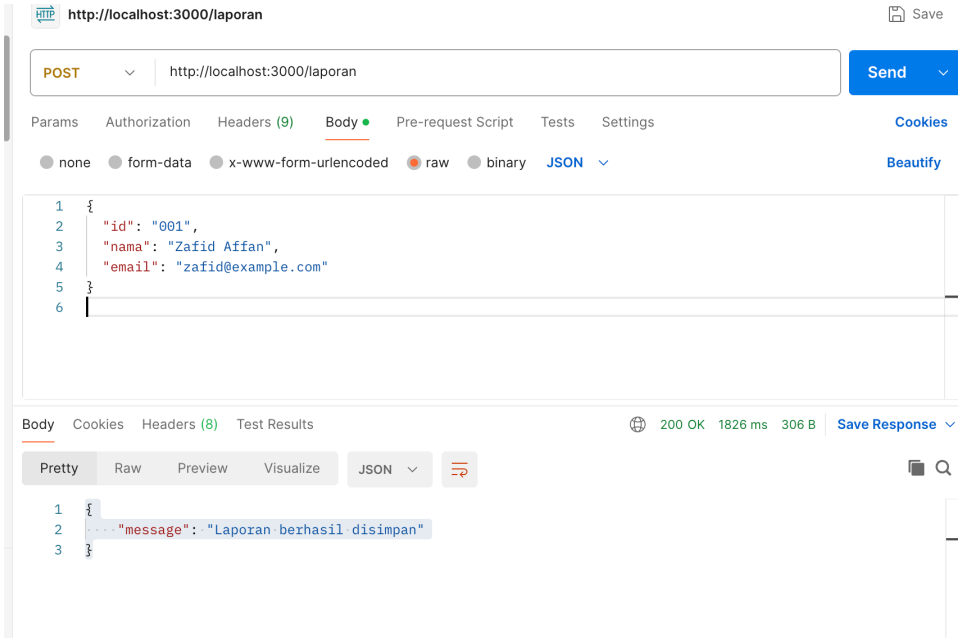
Di tab Body, pilih:

raw

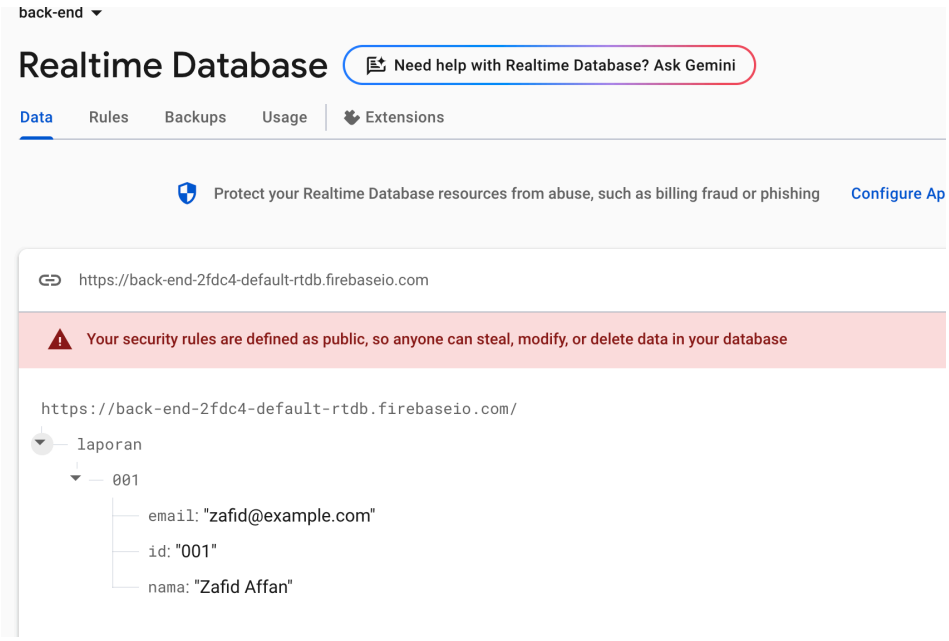
lalu di dropdown sebelah kanan pilih JSON



4. Klik Send



5. Cek di Database



F. Penjelasan API Endpoint

Endpoint: POST /laporan

Deskripsi: Menyimpan data laporan baru ke Firebase Realtime Database.

Request Body:

```
{
  "id": "string",
  "nama": "string",
  "email": "string"
}
```

Response:

- 200 OK

```
{
  "message": "Laporan berhasil disimpan"
}
```

- 400 Bad Request jika ada field yang kosong

```
{
  "message": "Semua field (id, nama, email) wajib diisi"
}
```

- 500 Internal Server Error jika terjadi kesalahan server

G. Kesimpulan

Melalui proyek ini, telah berhasil dibangun sebuah REST API sederhana menggunakan Node.js dan Express yang terhubung dengan Firebase Realtime Database sebagai penyimpanan data. Implementasi ini menunjukkan bahwa proses CRUD data laporan dapat dilakukan secara efisien dan real-time dengan memanfaatkan layanan cloud Firebase. Selain itu, pengujian menggunakan Postman membuktikan bahwa endpoint API yang dibuat berjalan sesuai dengan fungsinya, khususnya dalam menyimpan data laporan pengguna.

Penggunaan middleware seperti express.json() dan cors dalam Express juga memudahkan pengolahan data dari client serta membuka akses lintas domain, yang sangat berguna untuk pengembangan frontend di masa mendatang. Struktur proyek yang sederhana membuatnya

mudah dipahami dan dapat dijadikan dasar bagi pengembangan backend lebih lanjut, baik untuk aplikasi web maupun mobile.

Secara keseluruhan, proyek ini memberikan pemahaman praktis mengenai integrasi backend dengan layanan cloud database, serta menjadi langkah awal yang baik dalam membangun sistem pelaporan digital yang skalabel dan modern.